



Université Montpellier Paul-Valéry

Master MIASHS

Mémoire de stage

Polypbase

Conception d'une webapp de suivi des polypes de méduses

Aquarium de Paris

Suivi de laboratoire, données historiques et interface tablette

Réalisé par

Ayoub AKKOUH,
Anthony COMBES-AGUÉRA,
et Clément ROMANET

Tuteur universitaire

Sophie LEBRE

Tuteurs de stage

Étienne BOURGOUIN
Anaïs COURTET

Dépôt GitHub

Anthony2210/POLYPBASE

Année universitaire 2025-2026

Remerciements

Nous tenons à remercier Anaïs Courtet et Étienne Bourgouin pour leur accueil au sein de l'Aquarium de Paris et pour le temps qu'ils nous ont accordé depuis le début du stage. Leur disponibilité, leurs explications et leurs retours nous ont permis de mieux comprendre le fonctionnement du service des méduses et les besoins réels derrière Polybase. Nous les remercions aussi pour leur sympathie et pour la visite guidée de l'Aquarium. Cette découverte du lieu, des espaces ouverts au public et des coulisses nous a aidés à mieux situer notre travail. Polybase accompagne un travail quotidien en laboratoire, dans un environnement vivant, précis et déjà très organisé. Nous remercions Madame Sandra Bringay pour son aide sur la reprise du modèle conceptuel de données et pour les nombreux renseignements qu'elle nous a apportés sur le projet. Ses retours nous ont aidés à mieux structurer les entités importantes et à prendre plus de recul sur la manière de modéliser les données. Nous remercions également Monsieur Servajean pour ses conseils sur l'usage de Git, des branches et de l'organisation du dépôt. Ces remarques nous ont permis de corriger notre approche et de préparer une base plus propre pour la suite du stage. Enfin, nous remercions Madame Sophie Lebre, qui nous a transmis cette offre de stage et nous a soutenus pendant toute la durée du stage. Son partage nous a permis de rejoindre un projet concret, utile et directement lié à des problématiques de données, de suivi scientifique et de développement web.

Table des matières

1.	Introduction	3
2.	Contexte de l’Aquarium de Paris et du service méduses	4
3.	État de l’art	5
4.	Problématique et objectifs du stage	7
5.	Organisation du projet et méthode de travail	8
6.	Analyse des données historiques	9
6.1.	Structure des fichiers source	9
6.2.	Création du CSV complet	9
6.3.	Contrôles de qualité des données	10
6.4.	Périmètre du suivi	11
6.5.	Premiers constats exploitables	12
6.6.	Première exploration prédictive	14
7.	Conception et réalisation de Polypbase	16
7.1.	Partir des usages pour choisir l’architecture	16
7.2.	Organiser les données autour des boîtes et des structures	16
7.3.	Intégrer les droits et la confidentialité dans le fonctionnement	18
7.4.	Construire un parcours court pour le laboratoire	18
7.5.	Relier l’interface à une API contrôlée	21
7.6.	Préparer la traduction, les essais et la reprise du projet	21
7.7.	Délimiter clairement l’état actuel	22
8.	Bilan critique et perspectives	23
8.1.	Résultats obtenus	23
8.2.	Points à consolider	23
8.3.	Enseignements du stage	24
8.4.	Prochaines étapes	24
9.	Conclusion	25
A	Maquette papier	26

Résumé

Le service des méduses de l'Aquarium de Paris suit des cultures de polypes depuis plusieurs années à l'aide de fichiers Excel annuels. Ces fichiers conservent un historique précieux, mais leur forme rend la recherche, la comparaison entre années et le partage avec d'autres structures difficiles à mesure que le nombre de boîtes croît et que plusieurs personnes interviennent.

Ce stage a pour objectif de concevoir Polybase, une application web destinée à remplacer ce suivi dispersé. L'application doit permettre à un technicien de retrouver rapidement une boîte en laboratoire, de saisir un relevé sur tablette, de consulter l'historique sur ordinateur et de suivre dans quelles armoires les boîtes se trouvent. L'apparition d'éphyrules est un signal important pour le laboratoire. Elle indique que des polypes sont entrés en strobilation et peut conduire à un changement de température adapté à l'espèce, à son origine et à l'histoire de la souche. Une première exploration prédictive a été menée dans des notebooks à partir des relevés historiques. Elle cherche à déterminer si les comptages, la température et la période de l'année contiennent un signal utile pour repérer l'apparition d'éphyrules. Ce travail reste séparé de l'application et doit encore être consolidé avant tout usage opérationnel.

Le travail réalisé repose sur une architecture hybride Django et React. Le backend expose une API REST structurée autour des boîtes, des relevés biologiques, des zones thermiques et des profils utilisateurs. Le frontend React assure la navigation, la recherche, la saisie des relevés et l'adaptation à la tablette. Une analyse exploratoire des fichiers historiques, menée dans un notebook Python, a permis de comprendre la structure des données sources, d'identifier les ambiguïtés de saisie et de préparer la transformation vers un format long exploitable par l'application.

Le projet pose une première base technique et méthodologique. L'import des données historiques, la génération des QR codes, les exports filtrés, les actions de repiquage et la collecte des températures depuis les sondes constituent les principales étapes à finaliser avant une utilisation complète au laboratoire.

Mots-clés : application web, suivi de laboratoire, polypes de méduses, exploration prédictive, Django, React, données historiques, aquarium.

1. Introduction

Notre travail part d'un besoin très concret de l'Aquarium de Paris. Le service des méduses suit des cultures de polypes dans le temps, avec des espèces, des souches, des boîtes, des températures et des observations régulières. Ce suivi existe déjà sous forme de fichiers Excel. Ces fichiers ont une valeur importante, car ils conservent plusieurs années d'observations. Leur forme rend cependant la recherche, l'historisation, le partage et la reprise difficiles dès que le nombre de boîtes augmente ou que plusieurs personnes interviennent. La première ambition de Polypbase est de créer un outil commun entre plusieurs aquariums. Chaque structure doit pouvoir conserver ses propres données, tout en utilisant une manière partagée de nommer les espèces, de noter les boîtes, de saisir les relevés et d'échanger certaines informations. Cette cohérence est essentielle pour que les données puissent être comprises ailleurs qu'au laboratoire qui les a produites. Elle demande une gestion des organisations, des droits, des espèces, des souches, des transferts et des règles de partage. Le stage consiste à transformer ce suivi dispersé en une application web utilisable au laboratoire et sur ordinateur. L'outil doit permettre de retrouver rapidement une boîte, de saisir un relevé, de consulter l'historique, de suivre les différentes armoires qui sont utilisées à différentes températures, de préparer des exports et de garder une trace des actions. La boîte de culture reste le point de repère principal. Elle relie l'espèce, la souche, la localisation, la température, les relevés, les observations, les repiquages et les échanges éventuels entre structures. L'apparition d'éphyrules dans une boîte est un événement clé du cycle biologique. Elle signale que des polypes sont entrés en strobilation et peut conduire l'équipe à modifier les conditions thermiques de la culture. Dans le prolongement de l'analyse historique, deux notebooks exploratoires ont testé la possibilité de repérer ce phénomène à partir des comptages, de la température et de la période de l'année. Cette étude constitue une piste de travail. Elle n'est pas encore reliée à l'API ni à l'interface de Polypbase. Ce mémoire présente le contexte scientifique et professionnel du stage, puis l'état de l'art utilisé pour cadrer le projet. Il expose ensuite la problématique, l'organisation du travail, l'analyse des fichiers historiques, la conception et la réalisation de Polypbase, les choix liés aux droits et à la confidentialité, puis les limites et perspectives du projet.

2. Contexte de l’Aquarium de Paris et du service méduses

L’Aquarium de Paris est un lieu de présentation, de médiation et de travail scientifique autour du monde aquatique. Situé dans les Jardins du Trocadéro, il accueille le public tout en maintenant en coulisses des espaces techniques et des élevages spécialisés. Son Médusarium occupe une place importante dans cette identité. L’établissement présente une grande collection de méduses en Europe, avec des espèces montrées par roulement, et ses biologistes travaillent sur l’entretien, le maintien et la reproduction de ces animaux [2]. Le stage se déroule au contact du service des méduses. Ce service suit des cultures de polypes dans le temps. Pour comprendre l’intérêt de Polybase, il faut rappeler le cycle que l’application accompagne. La méduse correspond au stade libre et visible de l’animal. Le polype est un stade fixé, plus discret, qui peut rester dans une boîte de culture et produire de nouveaux individus. L’éphyrule est le jeune stade libéré lors du passage du polype vers la forme méduse. Dans la littérature scientifique, on parle souvent d’éphyra. Dans le laboratoire et dans les fichiers de suivi, le terme éphyrule est utilisé pour le comptage quotidien. Le passage du polype vers l’éphyrule est appelé strobilation. Helm décrit ce processus comme la métamorphose d’un polype en jeune méduse, avec une production possible d’une ou plusieurs éphyras selon les espèces [6]. Cette étape explique pourquoi le suivi doit dépasser le simple nombre de boîtes. Il faut savoir quelle souche est observée, dans quelle armoire elle se trouve, combien de polypes sont présents, combien d’éphyrules sont produites et quels événements ont eu lieu autour de la boîte. Les cultures suivies correspondent à des souches et à des espèces différentes. Elles sont placées dans des boîtes, puis maintenues dans des conditions contrôlées, notamment avec des températures différentes selon les besoins du suivi. Le rôle de l’équipe est d’observer régulièrement ces boîtes, de compter les polypes, de relever les éphyrules et de noter les événements importants. Ces informations servent à comprendre l’évolution des cultures, à garder un historique fiable et à préparer les décisions de laboratoire.

3. État de l’art

Le projet s’inscrit dans plusieurs familles de références. La première est celle de la numérisation d’un suivi de laboratoire. L’objectif est de rendre les données plus traçables, de conserver leur historique, de retrouver rapidement une boîte et de préparer une réutilisation scientifique ou réglementaire des relevés. La deuxième référence est celle des plateformes existantes pour aquariums. Fauniabase, développé par Luxaqua, est présenté comme un logiciel de gestion et de suivi des animaux pour les aquariums publics. Il met en avant la centralisation des données, le suivi des animaux et des bacs, la traçabilité, la planification de tâches et l’usage de QR codes [4]. Cette référence montre l’existence d’outils professionnels autour de la gestion d’inventaires vivants en aquarium. Fauniabase apporte une référence utile pour Polyppbase. Polyppbase cible toutefois un besoin plus spécialisé : les cultures de polypes de méduses, leurs relevés réguliers, leur historique, les repiquages, les conditions de température et l’usage en laboratoire sur tablette. Le travail retient surtout certaines idées utiles, comme la centralisation des données, la traçabilité, la recherche rapide et l’identification par QR code. ZIMS (Zoological Information Management System), développé par Species360, est utilisé par des centaines d’établissements zoologiques et aquariums dans le monde. Il couvre la gestion des dossiers individuels des animaux, les transferts entre institutions, le suivi médical, la reproduction et la généalogie [1]. Sa couverture internationale en fait une référence pour l’interopérabilité entre structures. ZIMS illustre cependant les difficultés à éviter pour Polyppbase. Conçu pour gérer des animaux individuels dans de grandes institutions, il rassemble un grand nombre d’informations avec une navigation complexe et de nombreuses références croisées. Cette étendue ne correspond pas au besoin du laboratoire, où l’unité de suivi est une boîte de culture et non un individu, et où l’interface doit rester utilisable rapidement sur tablette. L’application doit privilégier des codes clairs, des écrans courts et des actions évidentes. La troisième référence concerne la taxonomie. Une application utilisée par plusieurs structures doit éviter les ambiguïtés sur les noms d’espèces. Le World Register of Marine Species, ou WoRMS, donne accès à des noms valides, des synonymes, des noms vernaculaires et des identifiants stables pour les organismes marins [5]. Polyppbase peut ainsi relier ses espèces à des références reconnues sans porter toute la complexité taxonomique. C’est pour cette raison que le modèle **Species** prévoit un identifiant WoRMS éventuel. La quatrième référence vient de la biologie des méduses. Les articles sur les polypes et les éphyrules aident à justifier les données suivies. Helm rappelle que beaucoup de scyphozoaires ont un cycle comprenant une larve planula, un polype benthique et une méduse, et que la transformation du polype en éphyra s’appelle la strobilation [6]. Fuchs et ses coauteurs montrent aussi que, chez *Aurelia aurita*, la transition du polype vers la méduse est déclenchée par des signaux environnementaux et que des protéines sont fortement activées avant la métamorphose en réponse aux changements saisonniers de température [7]. La température apparaît donc comme une donnée importante à conserver. Une étude expérimentale sur trois espèces de scyphozoaires indique que la température peut jouer un rôle dans la reproduction asexuée des polypes, le moment de la strobilation et la production d’éphyras [8]. Ces résultats expliquent l’intérêt de relier les comptages biologiques aux zones thermiques, aux sondes, aux changements de température et aux historiques de boîtes. Le contexte scientifique de l’Aquarium donne aussi du sens au projet. Le LabCom Jellyfish and Polyps Academy explique que l’Aquarium de Paris maintient une grande collection de polypes de méduses et travaille avec MARBEC sur la culture des méduses, la création d’une banque de référence de polypes et la compréhension des paramètres qui influencent leurs proliférations [3]. Polyppbase s’inscrit dans cette logique. L’application doit aider à conserver des données plus propres, plus faciles à retrouver et plus adaptées à une analyse future.

Aucun de ces outils ne répond pleinement au besoin de l'Aquarium de Paris. Fauniabase s'approche le plus du contexte aquarium, mais reste centré sur la gestion générale des animaux et des bacs, sans adresser le suivi fin de cultures de polypes, les relevés hebdomadaires, les repiquages ou la prédiction d'éphyrules. ZIMS couvre un périmètre bien plus large mais impose une complexité d'usage incompatible avec un laboratoire sur tablette. WoRMS apporte la référence taxonomique mais ne constitue pas un outil de suivi. Polybase se positionne donc dans un espace non couvert : une application légère, spécialisée pour le suivi de cultures de polypes de méduses, conçue pour être utilisée au quotidien en laboratoire et partagée entre structures partenaires.



Figure 1 – Interface de ZIMS (Species360), exemple de plateforme de gestion animale pour aquariums et zoos.
Source : Species360, *ZIMS for Aquariums* [1].

4. Problématique et objectifs du stage

La problématique du stage peut se formuler ainsi : comment concevoir une application web maintenable capable de remplacer un suivi Excel historique, avec une saisie simple en laboratoire, une consultation claire sur ordinateur et une base commune utilisable par plusieurs aquariums. Cette problématique contient plusieurs contraintes. La première est métier. L'application doit respecter la façon dont le service des méduses suit ses cultures. Elle doit donc intégrer les boîtes, les souches, les espèces, les températures, les observations, les repiquages, les éphyrules et l'historique. La deuxième contrainte est ergonomique. Une tablette utilisée en laboratoire doit afficher peu d'informations et permettre une saisie rapide. La troisième contrainte est technique. Le projet doit rester compréhensible par des développeurs qui reprendront le code après le stage. La quatrième contrainte concerne la donnée. Les fichiers Excel anciens doivent être compris avant toute tentative d'import. Nous avons avancé par étapes. Il fallait d'abord cadrer le besoin et comprendre les données existantes. Il fallait ensuite structurer le projet avec une architecture claire, créer les premiers modèles Django, stabiliser une API propre et produire une première interface React orientée tablette et ordinateur. La suite du travail prépare aussi les données de démonstration, la documentation, le notebook d'analyse exploratoire et les scripts d'import ou de QR codes à compléter. Le projet reste en construction. Le travail réalisé pose une base technique et méthodologique. Les imports réels, les QR codes, les exports, l'administration métier complète et l'intégration des sondes doivent encore être finalisés.

5. Organisation du projet et méthode de travail

Nous avons organisé le travail autour d'échanges réguliers avec l'équipe de l'Aquarium de Paris. Anaïs Courtet et Étienne Bourgoignin nous ont accompagnés dans la compréhension du besoin métier. Ces échanges ont permis de clarifier les contraintes du laboratoire, de corriger certains choix de conception et de vérifier que les premiers écrans restaient cohérents avec l'usage attendu. Le projet a été organisé autour d'un dépôt GitHub public. Le code est séparé en dossiers fonctionnels afin de faciliter la lecture et la reprise. Le dossier **backend** contient Django, les modèles, les API et les tests. Le dossier **frontend** contient l'application React. Le dossier **docs** contient les notes, les maquettes et le MCD. Le dossier **notebooks** contient l'analyse exploratoire des fichiers Excel. Le dossier **scripts** contient les points d'entrée prévus pour l'import Excel et la génération de QR codes. Un espace Notion a aussi été utilisé pour garder les décisions, les commandes utiles et les notes de travail. Ce support facilite la reprise par les membres du groupe et par les personnes qui interviendront après le stage. Le README complète cette organisation en expliquant les commandes, la structure du projet, les endpoints API, les comptes de démonstration et la gestion des dépendances avec uv. La méthode suivie a consisté à construire d'abord la base du projet avant de pousser l'interface trop loin. Le modèle de données et l'API ont été priorisés, car le frontend dépend directement de la structure des données. L'interface React a ensuite servi à tester les usages principaux : recherche d'une boîte, consultation d'une fiche, saisie d'un relevé, affichage des zones thermiques et modification de la langue du profil. L'historique Git reflète cette progression par étapes. La structure générale et le notebook exploratoire ont été posés en premier. Les modèles ont ensuite été répartis dans les applications Django, puis l'API et la première interface React ont été ajoutées. Les itérations suivantes ont relié le formulaire de relevé à l'API, adapté la fiche au format tablette, introduit uv et ajouté les notebooks de modélisation exploratoire. Cette progression a permis de tester chaque couche avant d'élargir le périmètre. Cette organisation répond aussi à une contrainte de maintenance. Le tuteur de stage a indiqué que l'Aquarium pourra faire appel à des informaticiens pour la reprise. Le code doit donc rester lisible, commenté avec des phrases simples en anglais, et documenté suffisamment pour qu'un développeur extérieur comprenne rapidement le rôle de chaque partie.

6. Analyse des données historiques

6.1. Structure des fichiers source

Les fichiers de suivi historiques montrent comment les données sont réellement notées par l'équipe. Les fichiers `Suivi_2019.xlsx` à `Suivi_2026.xlsx` sont organisés par années. Dans ces fichiers, les feuilles correspondent à de grands groupes comme Aurelia, Hydrozoa, Cubozoa, Rhizostomae, Semaestomeae ou Coronatae. Certaines années ajoutent aussi d'autres feuilles, comme UMR_MARBEC ou Staurozoa. Le fichier `Suivi_2019.xlsx` est vide : l'analyse porte donc sur les années 2020 à 2026. Dans les feuilles annuelles, les premières colonnes décrivent l'espèce, le numéro de boîte et la température. Les colonnes suivantes correspondent aux semaines ou aux mois de suivi. Pour une même boîte, les données sont notées sur plusieurs lignes, avec une ligne pour le nombre de polypes et une autre pour le nombre d'éphyrules. La collecte suit donc une logique de tableau de laboratoire. L'équipe observe les boîtes, note les comptages dans la cellule correspondant à la période de suivi, puis conserve l'historique dans le même fichier annuel. Cette forme facilite la saisie dans Excel semaine après semaine. Elle devient cependant plus difficile à exploiter quand il faut retrouver une boîte précise, comparer plusieurs années ou préparer un import dans une base de données. Le dossier de données contient aussi des fichiers complémentaires. Un fichier est consacré aux repiquages et aux changements de température. Le fichier `SuiviStrobilation.xlsx` contient des informations liées à la strobilation, avec des colonnes comme l'espèce, la température, la salinité et des conditions expérimentales. Le fichier `A.limbata.xlsx` reprend une structure plus déjà normalisée, avec une colonne de date et des groupes de colonnes associant une boîte, les polypes, les éphyrules et la température. Ces différences demandent un import plus fin qu'un simple copier coller. Il faudra reconnaître plusieurs formes de fichiers.

6.2. Création du CSV complet

Nous avons créé le fichier `polyp_tracking_long_format.csv` avec le notebook afin de rassembler toutes les mesures dans un seul tableau. Dans ce format long, chaque ligne correspond à une mesure possible pour une boîte, une année, une semaine et un type de mesure. La colonne `file_year` vient du nom du fichier annuel. La colonne `group` vient de la feuille Excel. Les colonnes `species`, `box` et `temperature` reprennent les informations associées à la boîte. La colonne `measurement_type` conserve le libellé présent dans Excel, tandis que `normalized_measurement_type` simplifie ce libellé en deux familles principales, `polyps` et `ephyrae`. La colonne `week_index` indique la semaine reconstruite, `excel_column` garde la position de la colonne source et `value` contient la valeur numérique quand elle existe. Le CSV garde les cellules vides comme des absences de valeur. Cette décision est importante pour ne pas confondre une case vide avec un zéro saisi. Le zéro indique qu'un relevé a été fait avec une valeur nulle. Une cellule vide peut venir d'une absence de suivi, d'une boîte non active, d'un changement de température ou d'un repiquage. Ce fichier sert de support de travail pour comprendre l'import futur. Il ne remplace pas encore les modèles Django, mais il rapproche les données historiques d'une structure exploitable par l'application. Au total, le tableau consolidé rassemble 237 848 lignes décrivant 573 boîtes, 146 espèces et 8 groupes sur les années 2020 à 2026. Parmi ces lignes, 76 404 seulement portent une valeur numérique, ce qui confirme la part importante des cellules laissées vides dans les fichiers.

Variable	Description
<code>file_year</code>	Année du fichier Excel source, extraite du nom du fichier.
<code>group</code>	Nom de la feuille Excel, souvent lié à un groupe de méduses.
<code>species</code>	Nom de l'espèce écrit dans la feuille de suivi.
<code>box</code>	Identifiant de la boîte dans le fichier Excel.
<code>temperature</code>	Température indiquée dans la feuille Excel pour la boîte ou la ligne suivie.
<code>measurement_type</code>	Libellé original de la mesure, par exemple nombre de polypes ou nombre d'éphyrales.
<code>normalized_measurement_type</code>	Type de mesure simplifié pour l'analyse, avec polyps ou ephyrae .
<code>week_index</code>	Numéro de semaine reconstruit à partir des colonnes du tableau Excel.
<code>excel_column</code>	Position de la colonne source dans le fichier Excel.
<code>value</code>	Valeur numérique saisie, laissée vide quand aucune valeur n'est présente dans Excel.

Table 1 – Variables du fichier `polyp_tracking_long_format.csv`.
Source : table construite à partir de `notebooks/excel_eda.ipynb`.

6.3. Contrôles de qualité des données

Le notebook `notebooks/excel_eda.ipynb` documente cette première analyse exploratoire. Son rôle est de comprendre la structure des fichiers, d'identifier les colonnes, de repérer les valeurs réellement saisies et de préparer une transformation utilisable par l'application. Il insiste aussi sur un point important. Une cellule vide peut avoir un sens différent de zéro. Elle peut correspondre à une absence de suivi, à une boîte inactive, à une période avant ou après un changement de température, ou à un repiquage qui a modifié le suivi. Un zéro saisi et une absence de valeur portent donc deux informations différentes.

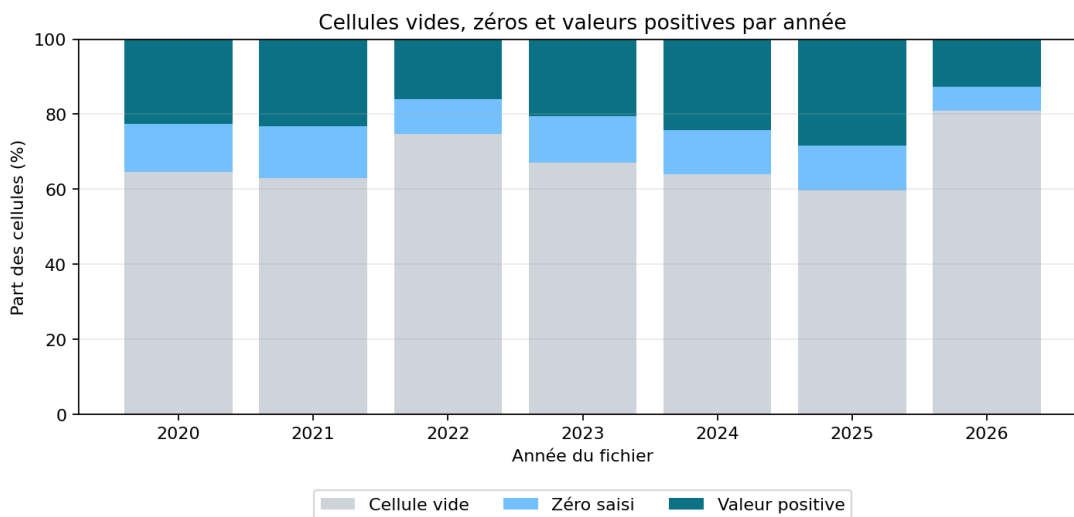


Figure 2 – Répartition des cellules vides, des zéros saisis et des valeurs positives par année.
Source : figure générée depuis `notebooks/excel_eda.ipynb`.

La figure montre que les cellules vides occupent une part importante des tableaux. Ce constat oriente directement la conception de l'import. Il ne faudra pas transformer automatiquement ces vides en zéros. Le notebook produit aussi une figure de couverture hebdomadaire. Elle permet de repérer les périodes où les

fichiers sont moins remplis et les années qui ne couvrent pas toutes les semaines de la même façon.

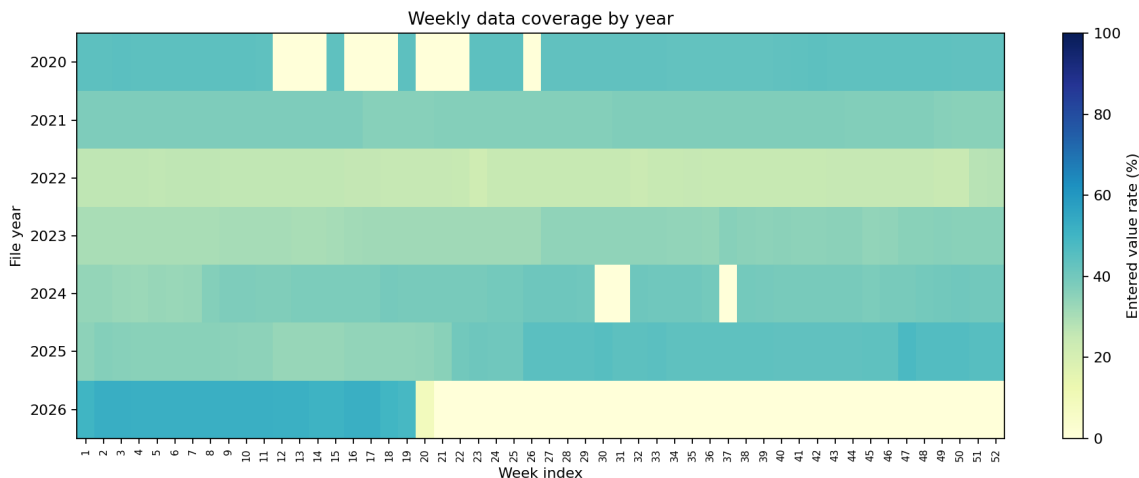


Figure 3 – Taux de valeurs saisies par semaine et par année.
Source : figure générée depuis `notebooks/excel_eda.ipynb`.

6.4. Périmètre du suivi

Les figures issues du notebook montrent aussi que le périmètre de suivi varie selon les années. Le nombre de boîtes présentes dans les fichiers augmente jusqu'en 2025, puis diminue dans le fichier 2026 disponible au moment de l'analyse. Cette observation aide à rester prudent dans les comparaisons entre années. Une variation peut venir d'un changement biologique, mais aussi d'une différence de périmètre, de durée de suivi ou de saisie.

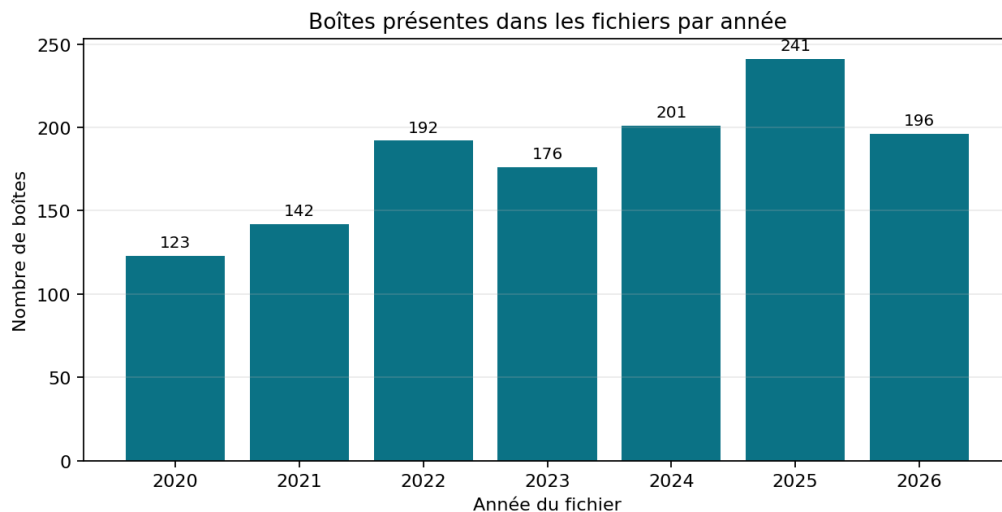


Figure 4 – Nombre de boîtes présentes dans les fichiers de suivi par année.
Source : figure générée depuis `notebooks/excel_eda.ipynb`.

Le nombre d'espèces présentes varie aussi selon les fichiers. Ce point justifie l'importance d'une gestion claire des espèces et des souches dans Polybase. Une application multi-structures doit éviter de mélanger une

variation réelle du suivi avec une variation liée à la manière dont les noms sont écrits ou organisés dans les fichiers.

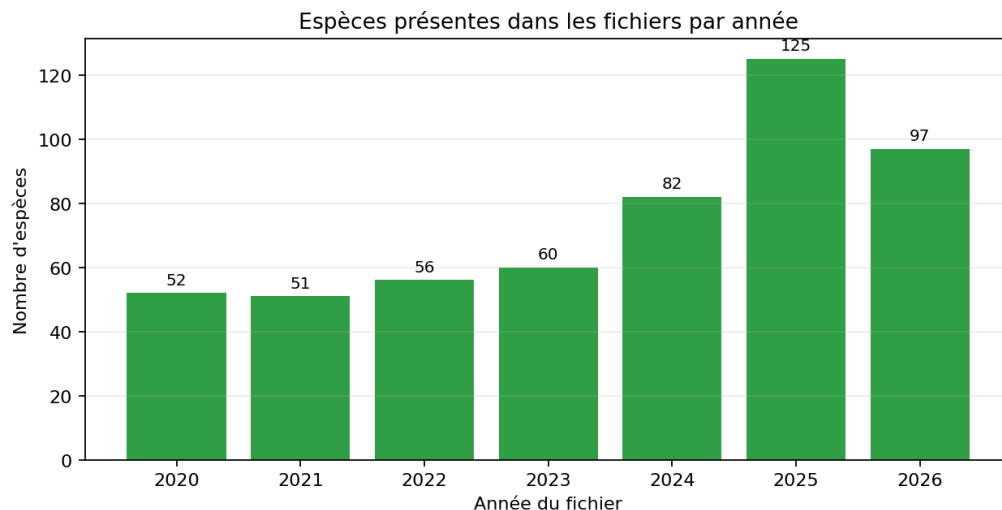


Figure 5 – Nombre d'espèces présentes dans les fichiers de suivi par année.
Source : figure générée depuis `notebooks/excel_eda.ipynb`.

6.5. Premiers constats exploitables

L'analyse ne sert pas seulement à préparer un import technique. Elle aide aussi à comprendre quels écrans seront utiles dans l'application. La durée de suivi des boîtes montre que certaines cultures apparaissent sur de nombreuses semaines. Cela confirme l'intérêt d'une fiche boîte avec un historique accessible rapidement.

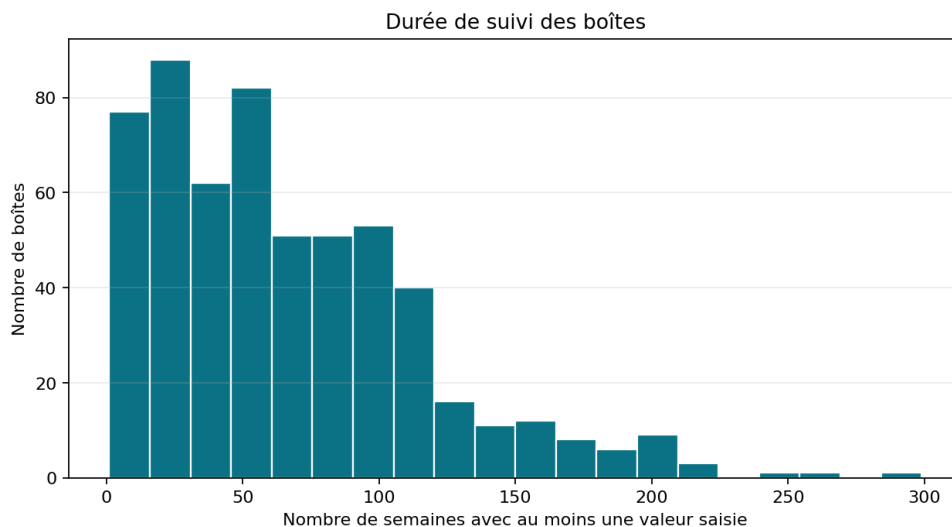


Figure 6 – Durée de suivi des boîtes, estimée à partir des semaines avec une valeur saisie.
Source : figure générée depuis `notebooks/excel_eda.ipynb`.

Le notebook repère aussi les espèces les plus suivies. Cette figure ne dit pas quelles espèces sont les plus importantes biologiquement. Elle montre plutôt quelles espèces pèsent le plus dans les fichiers disponibles. Cette information est utile pour choisir les premiers cas de test de l'import et de l'interface.

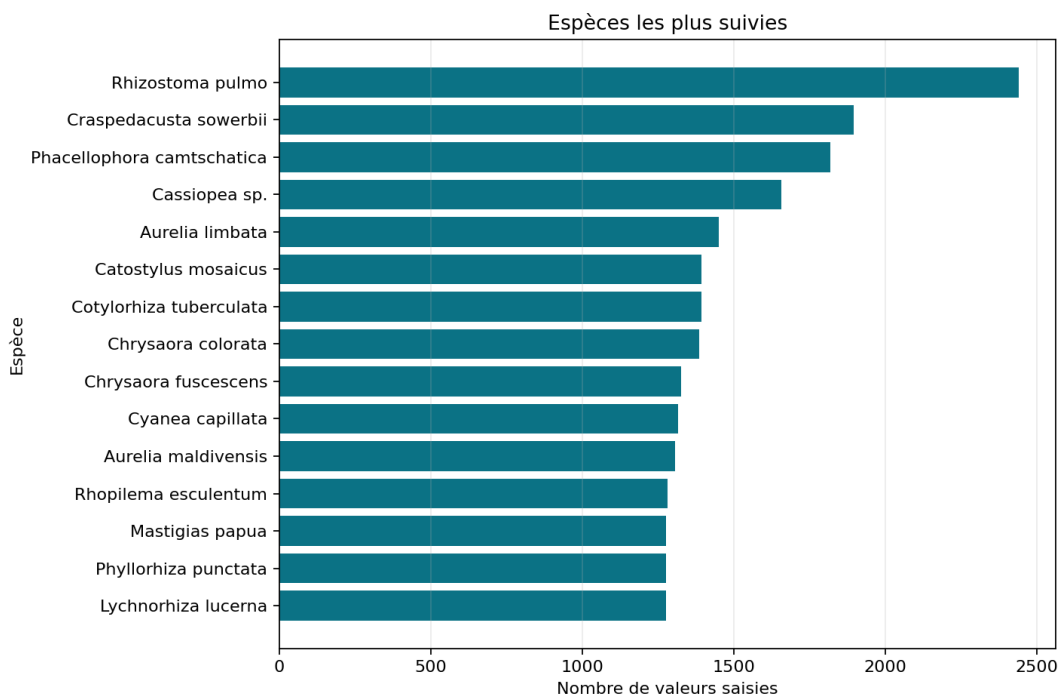


Figure 7 – Espèces les plus représentées dans les valeurs saisies.

Source : figure générée depuis `notebooks/excel_eda.ipynb`.

Une dernière lecture exploratoire compare la température indiquée dans Excel et la moyenne d'éphyrules pour quelques espèces bien représentées. Cette figure doit être interprétée avec prudence. Les données ne suffisent pas à isoler un effet biologique de la température, car les espèces, les boîtes, les périodes et les conditions de suivi changent aussi. Elle montre cependant pourquoi Polyibase doit relier les relevés à une armoire, à des changements de température et à un historique de boîte.

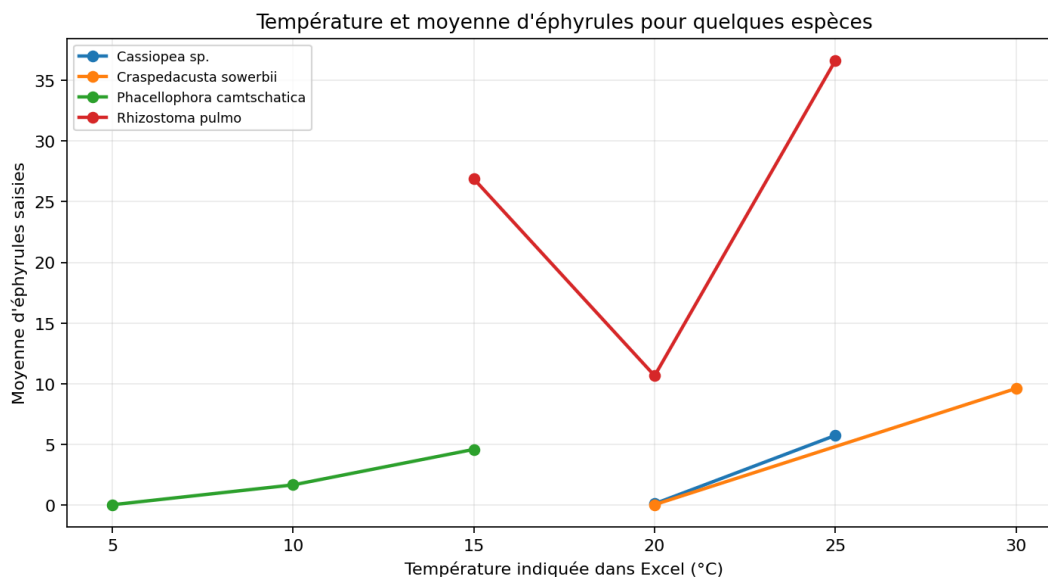


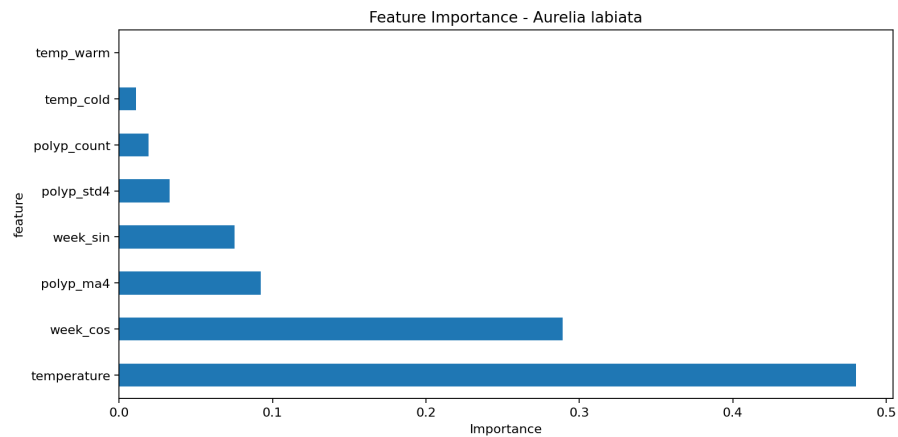
Figure 8 – Lien exploratoire entre température indiquée et moyenne d'éphyrules pour quelques espèces.

Source : figure générée depuis `notebooks/excel_eda.ipynb`.

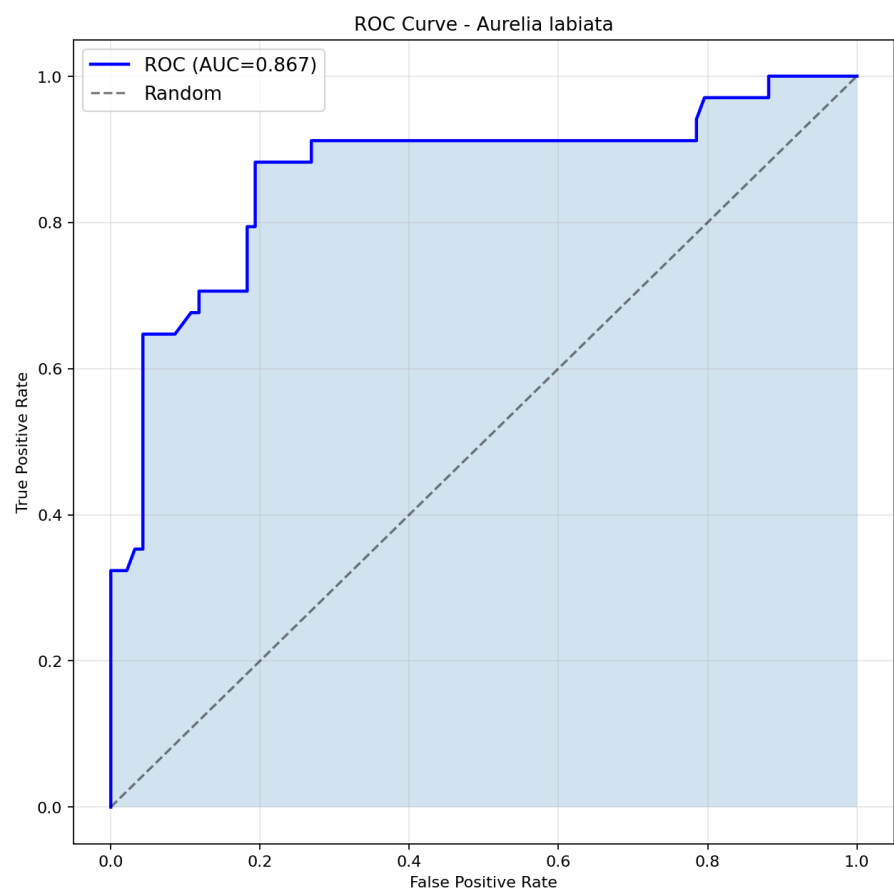
Cette analyse joue un rôle important dans la conception de Polypbase. Elle montre que l'application doit stocker les relevés dans un format plus structuré que les fichiers Excel. Elle montre aussi que l'import devra être validé avec l'équipe de l'Aquarium, car certaines décisions demandent une confirmation métier. Le notebook sert donc de pont entre les fichiers historiques et le futur import serveur.

6.6. Première exploration prédictive

Le CSV consolidé a aussi servi à tester une première question de modélisation. Nous avons cherché à savoir si les informations disponibles pouvaient aider à repérer les relevés associés à la présence d'éphyrules. Deux notebooks ont été consacrés à cette exploration sur *Aurelia labiata*, l'espèce retenue car elle disposait d'un historique suffisamment fourni dans les fichiers disponibles. Le notebook simplifié construit 635 observations hebdomadaires à partir de huit boîtes suivies entre 2020 et 2026. Parmi ces observations, 168 contiennent au moins une éphyrule. Les variables utilisées comprennent le nombre de polypes, la température, la position dans l'année et des statistiques glissantes calculées sur les comptages de polypes. Un classifieur par gradient boosting est ensuite entraîné sur un découpage aléatoire stratifié. Sur le jeu de test, le notebook indique une aire sous la courbe ROC de 0,867 et un rappel de 0,65 pour la classe correspondant à la présence d'éphyrules. Ces résultats montrent qu'un signal statistique existe dans cet échantillon. Ils ne suffisent pas à valider un outil d'aide à la décision. Le découpage utilisé est aléatoire et ne sépare ni les boîtes ni les années. Des observations provenant d'une même boîte peuvent donc se retrouver dans l'entraînement et dans le test. Le second notebook devait aussi intégrer les événements de repiquage et de changement de température, mais son exécution enregistrée indique qu'aucun marqueur d'événement n'a été chargé pour l'espèce étudiée. L'échantillon reste limité à huit boîtes et les données historiques contiennent les ambiguïtés décrites précédemment. Cette exploration doit donc être lue comme une preuve de faisabilité technique. Aucun score prédictif n'est actuellement exposé par l'API ou affiché dans la webapp. Une validation plus solide demanderait des données nettoyées, des événements mieux reliés aux relevés, une séparation temporelle ou par boîte et une discussion avec l'équipe du laboratoire sur l'usage réel d'une éventuelle alerte.



(a) Importance calculée des variables



(b) Courbe ROC sur le jeu de test

Figure 9 – Résultats du premier modèle exploratoire sur *Aurelia labiata*.

Source : figures générées depuis `notebooks/ml_ephyrae_simple.ipynb`.

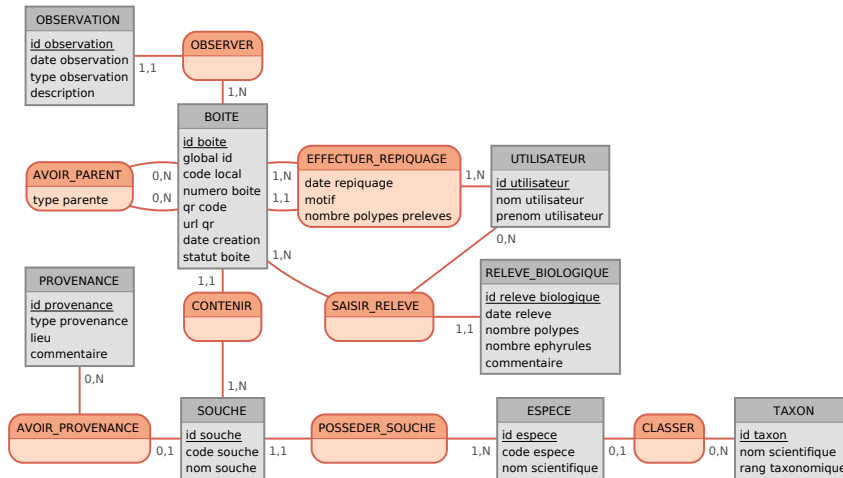
7. Conception et réalisation de Polybase

7.1. Partir des usages pour choisir l'architecture

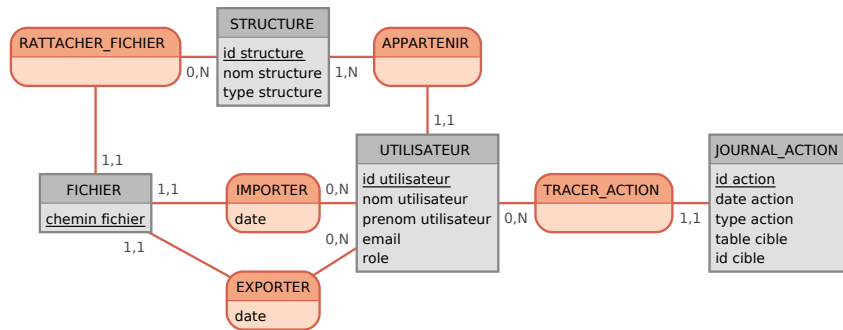
Nous avons retenu une architecture hybride après avoir distingué les deux contextes d'utilisation. Au laboratoire, le technicien doit retrouver une boîte et saisir un relevé rapidement sur une tablette. Sur ordinateur, les besoins portent davantage sur la consultation, l'administration, les exports et la préparation des données. Ces usages partagent la même base métier, mais ils n'imposent pas la même interface. Django prend en charge les modèles, l'authentification, les permissions, l'administration et les API. React porte l'application principale et les interactions qui demandent plus de fluidité. Ce partage permet de garder les règles sensibles dans le backend tout en adaptant précisément l'affichage à la tablette et à l'ordinateur. Le frontend ne décide donc pas seul si un utilisateur peut accéder à une boîte ou créer un relevé. Il demande les données au backend, qui applique les règles avant de répondre. Le backend et le frontend restent réunis dans un même dépôt. Ce choix facilite le suivi des évolutions qui touchent à la fois l'API et l'interface. Le dossier **backend** contient Django, le dossier **frontend** contient React et les dossiers **docs**, **notebooks** et **scripts** conservent les éléments nécessaires à la compréhension du projet. La configuration de la base accompagne aussi les différentes étapes du développement. En local, Django utilise SQLite si aucune variable `POSTGRES_DB` n'est définie. Lorsque les variables PostgreSQL sont présentes, la même application utilise PostgreSQL avec **psycopg**. SQLite simplifie les essais sur les postes de développement. PostgreSQL reste la solution visée pour une base partagée et déployée.

7.2. Organiser les données autour des boîtes et des structures

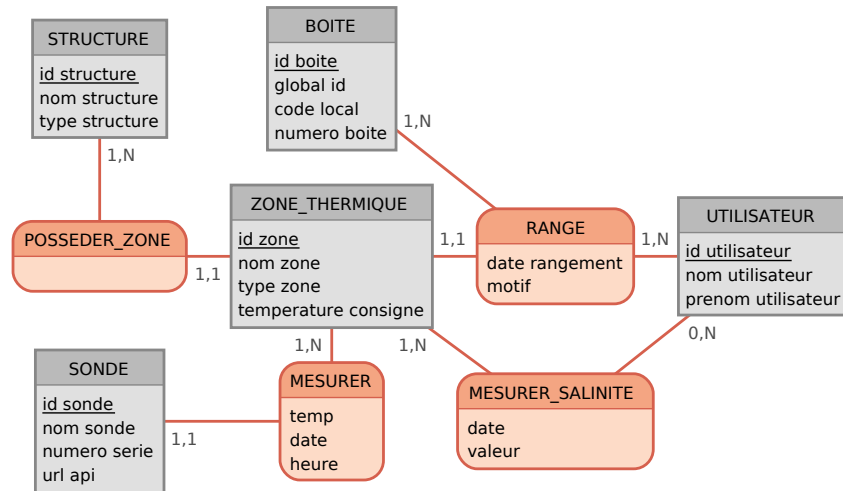
Le modèle de données a été travaillé avant de poursuivre les écrans. Cette étape était nécessaire pour relier correctement les espèces, les souches, les boîtes, les relevés, les températures et les organisations. Le MCD a été repris avec l'aide de Madame Sandra Bringay. La version actuelle est divisée en trois volets afin de rendre les relations plus lisibles. Le volet métier décrit principalement les boîtes, les souches, les espèces, les relevés et la parenté. Le volet gestion porte sur les structures, les utilisateurs, les fichiers et le journal des actions. Le volet environnement décrit les zones thermiques, les sondes, les rangements et les mesures.



(a) MCD métier



(b) MCD gestion



(c) MCD environnement

Figure 10 – Version de travail du modèle conceptuel de données.
Source : schémas réalisés avec Mocodo.

Cette organisation se retrouve dans les applications Django. **organizations** représente les aquariums, les partenaires et les accords de partage. **accounts** relie les utilisateurs à leurs organisations et à leurs rôles. **taxonomy** contient les taxons, les espèces, les origines et les souches. **cultures** regroupe les boîtes, les zones

thermiques, les localisations, les mouvements, les repiquages, la parenté, les identifiants et les transferts. `measurements` contient les relevés biologiques, les observations, les sondes, les températures et les salinités. `exports` et `audit` préparent les imports, les exports, les alertes et la traçabilité. La boîte constitue le point de liaison principal dans l’usage quotidien. Elle appartient à une organisation, correspond à une souche et se trouve dans une armoire avec une température définie pour celle-ci. Ses relevés sont datés et associés à l’utilisateur qui les a saisis. Le modèle conserve aussi les structures nécessaires pour enregistrer ses changements de localisation, ses relations de parenté et un éventuel transfert vers un autre aquarium. Le besoin multi-structures a influencé le modèle dès le départ. Le champ `global_code` fournit un identifiant commun, tandis que `local_code` permet de conserver une notation propre à chaque établissement. Les modèles `Organization`, `OrganizationMembership`, `SharingAgreement` et `BoxTransfer` donnent une base pour séparer les données locales et préparer des échanges. Les règles complètes de partage et le parcours de transfert ne sont pas encore reliés à l’interface.

7.3. Intégrer les droits et la confidentialité dans le fonctionnement

La gestion des droits a été placée dans le backend. Trois rôles sont définis dans les appartenances aux organisations : administrateur, technicien de laboratoire et lecteur. Les requêtes qui renvoient les boîtes et les zones thermiques sont filtrées selon les organisations accessibles à l’utilisateur connecté. Un utilisateur rattaché à l’Aquarium de Paris ne reçoit donc pas automatiquement les boîtes d’une autre structure. La création d’un relevé applique une vérification supplémentaire. Les administrateurs et les techniciens peuvent saisir les données du laboratoire. Le rôle en lecture seule reçoit une réponse d’interdiction lorsqu’il tente d’envoyer un relevé. Des tests automatiques contrôlent le filtrage par organisation, l’accès au détail d’une boîte et le refus d’écriture pour un lecteur. L’authentification de l’API repose actuellement sur la session Django. Les requêtes de modification passent aussi par la protection CSRF. Ce choix évite de mettre en place un second système de comptes pour React et permet de conserver l’administration Django. Il correspond au fonctionnement local actuel. Les réglages de production devront encore définir précisément les domaines autorisés, le secret Django, le mode de débogage et les paramètres HTTPS. Les données personnelles restent limitées aux informations utiles au compte, comme le nom d’utilisateur, l’adresse électronique lorsqu’elle est renseignée, le rôle, l’organisation et la langue choisie. Les données biologiques décrivent les cultures et les actions du laboratoire. Une première page de politique de confidentialité présente les finalités du traitement, les catégories de données, les droits des utilisateurs et les mesures attendues en production. La traçabilité a aussi commencé à être intégrée au parcours réel. Lorsqu’un relevé est créé ou modifié par l’API, une entrée est ajoutée dans `AuditLog` avec l’organisation, l’utilisateur, la boîte, l’action et la date. Le modèle prévoit d’autres actions comme le scan, le repiquage, le transfert, l’import et l’export. À ce stade, toutes ces actions ne sont pas encore reliées à un traitement fonctionnel. Ces choix répondent à la fois à une exigence de confidentialité et à un besoin d’organisation du travail. Ils permettent de savoir qui a saisi une donnée et de limiter les informations visibles selon la structure. Les durées de conservation des comptes, la procédure de suppression, les sauvegardes et la restauration de la base restent à définir avant le déploiement [9, 10].

7.4. Construire un parcours court pour le laboratoire

La maquette papier présentée en annexe a été construite sur place avec Étienne Bourguin et Anaïs Courtet, pendant notre séjour à l’Aquarium. Réalisée en commun avec l’équipe, elle nous a servi de base pour mettre les actions dans un ordre compréhensible avant de travailler le style visuel. Elle a permis de retenir un

parcours court autour de la recherche, de la fiche boîte, du dernier relevé et de la saisie. Les écrans React ont ensuite été repris plusieurs fois à partir des retours formulés pendant le développement. L'écran de pilotage sert maintenant de point d'entrée. La version actuelle permet de rechercher une boîte par son code, son espèce ou sa souche. Elle affiche des accès récents et des suggestions afin d'éviter une longue liste générale. La zone d'accueil est aussi destinée au futur scan de QR code, mais l'accès à la caméra et la lecture d'un code ne sont pas encore développés.

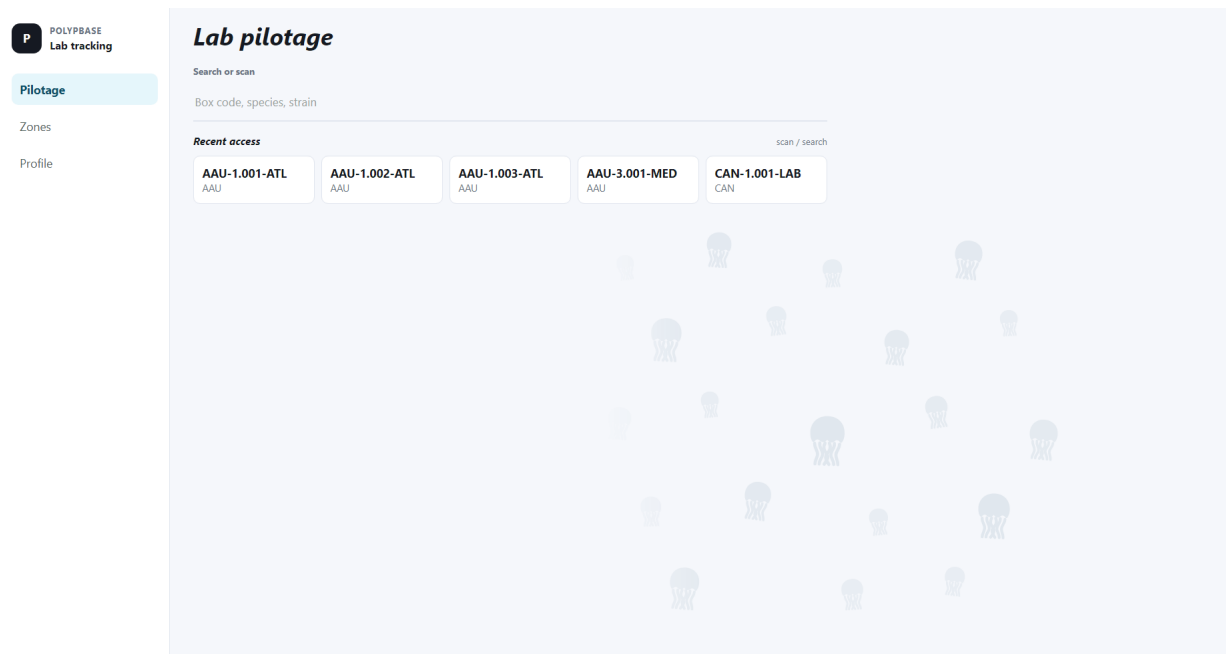


Figure 11 – Premier écran de pilotage dans l'interface React.
Source : capture réalisée depuis l'interface React de Polybase.

La fiche boîte rassemble les informations nécessaires avant une saisie. Elle affiche le code global, l'espèce, l'état de la boîte, l'armoire, l'organisation et le dernier relevé. Le formulaire contient les nombres de polypes et d'éphyrules, le dernier commentaire disponible et une zone d'observation. Des boutons ajoutent rapidement 50 ou 100 polypes et 10 ou 25 éphyrules. Sur tablette, la date du jour est utilisée sans afficher un champ supplémentaire. L'historique complet s'ouvre dans une fenêtre depuis la bande supérieure, ce qui libère de la place pour la saisie. La version ordinateur conserve une présentation plus développée. Elle affiche le dernier relevé, le formulaire et une partie de l'historique dans la page. Cette différence est gérée avec les mêmes données et le même composant React, puis adaptée par les règles de mise en page.

POLYPBASE
Lab tracking

Pilotage

Zones

Profile

[Back to pilotage](#)

Box sheet

AAU-1.001-ATL

active

Aurelia aurita

Cabinet 15 C
Aquarium de Paris

Last measurement 2026-05-29

Polyps: 100

Ephyrae: 10

New measurement 1 juin 2026

Measurement date: 01/06/2026

Polyps: 0

Ephyrae: 0

+50 +100 +10 +25

Last comment: Weekly demo measurement.

Observation: Quick lab note

Save measurement

(a) Version web

POLYPBASE
Suivi laboratoire

Pilotage

Zones

Profil

[Retour au pilotage](#)

AAU-1.002-ATL

active

Aurelia aurita

Dernier relevé 27 avr. 2026

Polypes: 99

Éphyrules: 4

Cabinet 15 C
Aquarium de Paris

Voir relevés 4

Nouveau relevé 1 juin 2026

Polypes: 0

Éphyrules: 35

+50 +100 +10 +25

Dernier commentaire: Weekly demo measurement.

Observation: Note rapide pour le laboratoire

Enregistrer le relevé

(b) Version tablette

Figure 12 – Fiche boîte sur ordinateur et adaptation en paysage pour tablette.
Source : captures réalisées depuis l'interface React de Polybase.

Les onglets Zones et Profil complètent cette première interface. La page Zones affiche les zones accessibles, leurs sondes et les dernières valeurs disponibles. La page Profil affiche le compte, les organisations autorisées et le choix entre le français et l'anglais. Cette première version permet ainsi de tester un parcours complet depuis la recherche d'une boîte jusqu'à l'enregistrement d'un relevé.

7.5. Relier l'interface à une API contrôlée

Le frontend React communique avec une API construite avec Django REST Framework. Les anciens endpoints prototypes en français ont été remplacés par des routes aux noms anglais, plus stables, qui renvoient des objets structurés par des sérialiseurs. Cette base commune garantit que l'interface et le serveur manipulent les mêmes représentations des boîtes, des relevés et des zones thermiques.

Endpoint	Rôle
GET /api/health/	Vérifie que le service répond. Cette route est publique.
GET /api/dashboard/	Renvoie les données de synthèse utilisées sur le premier écran React.
GET /api/boxes/	Renvoie la liste paginée des boîtes, avec une recherche et des filtres par statut ou organisation.
GET /api/boxes/<id>/	Renvoie une boîte avec son dernier relevé, son historique, ses alertes, ses identifiants et sa parenté.
GET /api/boxes/<id>/measurements/	Renvoie l'historique paginé des relevés biologiques d'une boîte.
POST /api/boxes/<id>/measurements/	Crée ou met à jour le relevé d'une boîte pour une date donnée.
GET /api/thermal-zones/	Renvoie les zones thermiques, les sondes et les dernières mesures disponibles.
GET /api/profile/	Renvoie le profil, les organisations accessibles et la langue choisie.
PATCH /api/profile/	Enregistre une nouvelle préférence de langue.

Table 2 – Endpoints déclarés dans la configuration actuelle de l'API.

Le parcours de saisie relie plusieurs éléments du projet. React prépare les valeurs du formulaire et les envoie à l'endpoint de la boîte. Le sérialiseur vérifie le format des données. Le backend retrouve ensuite la boîte parmi celles que l'utilisateur peut consulter, puis contrôle son droit d'écriture dans l'organisation concernée. La méthode `update_or_create` crée le relevé du jour ou met à jour celui qui existe déjà pour cette boîte et cette date. L'interface recharge enfin la fiche afin d'afficher les données enregistrées et un message de confirmation. Les tests couvrent les points les plus sensibles de cette première API. Ils vérifient l'accès public à la route de santé, la suppression des anciennes routes, la pagination, le cloisonnement entre organisations, le détail d'une boîte, la création d'un relevé, le refus pour un lecteur, les informations des zones thermiques et la mise à jour de la langue. Cette couverture reste principalement centrée sur le backend. Le frontend ne possède pas encore de tests automatisés.

7.6. Préparer la traduction, les essais et la reprise du projet

Le français reste la langue par défaut, l'anglais étant proposé comme seconde langue. Ce choix est enregistré dans le modèle `UserPreference` et reste modifiable depuis le profil, si bien qu'une même préférence pilote l'ensemble de l'application. Du côté serveur, les pages Django s'appuient sur leur système de traduction et les fichiers `.po` et `.mo`; du côté client, React puise ses textes dans deux dictionnaires français et anglais. Comme les deux parties lisent la même préférence renvoyée par l'API, la langue change partout sans qu'il faille maintenir deux interfaces distinctes. Une commande `seed_demo_data` prépare un environnement d'essai reproductible. Elle crée trois comptes correspondant aux rôles administrateur, technicien et lecteur, puis ajoute deux organisations et des exemples de taxonomie, de boîtes, de zones, de sondes, de relevés, d'observations, d'alertes, de parenté et de transferts. La commande utilise des mises à jour contrôlées afin de

pouvoir être relancée sans multiplier les données de démonstration. Les dépendances Python sont gérées avec `uv`. Le fichier `pyproject.toml` décrit les bibliothèques utilisées et `uv.lock` fixe leurs versions résolues. Le README explique la création de l’environnement, les migrations, les tests, le chargement des données de démonstration et le lancement de Django et React. Des pages Notion complètent ces indications avec les commandes courantes et les choix liés à la traduction. Nous avons aussi fixé une règle simple pour la reprise du code. Les noms de variables, les identifiants et les commentaires techniques utilisent un anglais simple. Les textes visibles par les utilisateurs restent traduits. Les vrais fichiers de l’Aquarium et la base locale sont exclus du dépôt. Cette organisation réduit le risque de publier des données de travail et aide un futur développeur à retrouver les éléments nécessaires sans dépendre de nos environnements personnels.

7.7. Délimiter clairement l’état actuel

Le projet contient déjà les modèles nécessaires à plusieurs fonctions futures. Les localisations, les mouvements de boîtes, les repiquages, la parenté, les identifiants QR, les transferts, les accords de partage, les sondes, les anomalies, les imports et les exports peuvent être administrés ou représentés dans la base. Les données de démonstration utilisent une partie de ces modèles. Leur présence dans la base ne signifie pas que tous les parcours sont terminés. Le repiquage ne possède pas encore d’action dédiée dans React. Les accords de partage ne sont pas encore appliqués par les permissions de l’API. Les enregistrements d’import et d’export existent, mais les services qui produiront les fichiers restent à développer. Le script de génération des QR codes est encore un point d’entrée vide. L’intégration de la sonde iMinilide a été étudiée dans une note technique, mais le format XML réel doit être observé sur l’appareil avant d’écrire le collecteur. Cette distinction est importante pour la suite du stage. La conception prépare un système plus large, tandis que la réalisation fonctionnelle actuelle se concentre sur la connexion, la recherche, la consultation d’une boîte, la saisie d’un relevé, les zones thermiques et la langue du profil. Ce périmètre forme un premier socle testable. Il permet de recueillir des retours concrets avant de développer les échanges entre aquariums et les automatisations. La durabilité de Polyphase dépendra autant de cette progression maîtrisée que du choix des technologies. La structuration des données peut préserver l’historique des cultures et faciliter sa transmission. Le cloisonnement par organisation peut soutenir une coopération entre aquariums sans confondre leurs données. L’interface adaptée à la tablette peut réduire les manipulations inutiles pendant la saisie. Ces bénéfices devront être confirmés par l’usage réel et accompagnés par des règles de conservation, des sauvegardes et une maintenance identifiée.

8. Bilan critique et perspectives

8.1. Résultats obtenus

Le projet permet désormais de tester un premier parcours cohérent. Un utilisateur peut se connecter avec une session Django, accéder aux données de ses organisations, rechercher une boîte, consulter son dernier relevé, afficher son historique et enregistrer les nombres de polypes et d'éphyrules avec une observation. Les zones thermiques et les informations de profil sont aussi accessibles depuis React. Le rôle en lecture seule est bloqué lors d'une tentative de saisie et chaque création ou modification de relevé ajoute une trace dans le journal. Cette réalisation reste limitée, mais elle relie déjà les différentes couches du projet. Les modèles Django, les sérialiseurs, les permissions, les endpoints, le client React et le formulaire travaillent sur les mêmes objets. Les données de démonstration rendent ce parcours reproductible sans utiliser les fichiers réels de l'Aquarium. L'analyse exploratoire apporte en parallèle une première compréhension des données historiques et de leurs difficultés d'import.

8.2. Points à consolider

Le modèle métier doit encore être validé. Le MCD version 2 a amélioré la séparation des domaines, mais certaines décisions restent ouvertes autour des rangements, des localisations, des repiquages, des codes et des règles de partage. Le backend conserve aussi des champs issus d'une étape antérieure, comme le nombre de strobiles, l'état de culture et l'indicateur de vigilance. Ils sont encore présents dans le modèle et dans les sérialiseurs, alors que le formulaire React actuel ne demande plus ces informations. Cette différence doit être tranchée avec l'équipe du laboratoire avant d'importer les vraies données. Le frontend a grandi rapidement pendant les itérations sur ordinateur et tablette. Une grande partie de l'application reste regroupée dans `App.tsx` et le fichier de style contient de nombreuses règles responsives. Cette organisation a permis de faire évoluer vite le prototype. La prochaine étape de maintenance devra séparer les vues, les composants de formulaire, la navigation et les fonctions de traduction sans modifier le comportement validé par les utilisateurs. Les tests automatiques actuels concernent surtout le backend. Ils contrôlent les permissions et les principaux endpoints, mais aucun test frontend ne vérifie encore la recherche, la navigation, la saisie ou le changement de langue. Les essais visuels ont été réalisés avec un format simulé dans les outils du navigateur. Le parcours doit encore être testé sur une Samsung Galaxy Tab Active 5 en orientation paysage, dans les conditions réelles du laboratoire. L'import historique reste un chantier sensible. Le format long facilite l'analyse, mais les cellules vides, les zéros, les changements de température et les repiquages ne peuvent pas être interprétés automatiquement sans règles validées. Le script `scripts/import_excel.py` ne réalise encore aucun import. Une première reprise devra utiliser un petit échantillon contrôlé et comparer chaque résultat aux fichiers sources. La même prudence s'applique aux QR codes, aux sondes et au modèle prédictif. Les identifiants QR sont prévus dans la base, mais leur génération et leur lecture restent à développer. Les modèles de températures existent, mais le collecteur iMinilide dépend du XML réel de la sonde. Les notebooks de prédiction montrent une piste statistique, mais leur évaluation ne permet pas encore de produire une alerte fiable dans l'application.

8.3. Enseignements du stage

L’exploration prédictive a aussi clarifié une limite de fond. Vouloir un modèle unique, capable de prédire l’apparition d’éphyrules pour toutes les familles de polypes à la fois, paraît aujourd’hui presque impossible. Chaque famille réagit différemment à la température et aux conditions de culture, les historiques sont hétérogènes et les fichiers comportent les ambiguïtés déjà décrites, à commencer par le fichier de suivi 2019 entièrement vide. Une piste plus réaliste pour la suite serait de viser d’abord un modèle sur la production de polypes, par famille ou par souche, plutôt qu’une alerte universelle sur les éphyrules. Le stage a aussi confirmé l’intérêt d’un découpage net des tâches. En séparant clairement l’analyse des données, le backend et l’API, puis l’interface React, chaque membre du groupe a pu avancer sur sa partie sans dépendre en permanence des autres. Cette organisation a apporté de l’agilité sur une durée courte et a limité les blocages, à condition de garder une base de données et une API communes comme point de rencontre.

8.4. Prochaines étapes

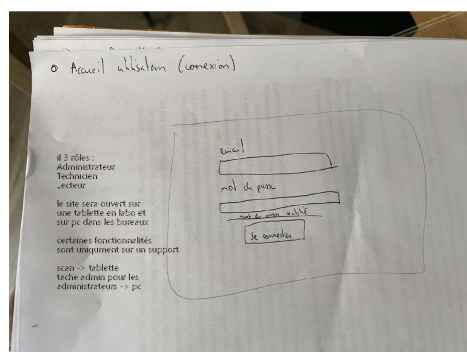
La prochaine priorité fonctionnelle est le repiquage. Cette action doit créer une nouvelle boîte à partir d’une boîte parent, reprendre les informations utiles, conserver le lien de parenté et enregistrer l’utilisateur, la date et le motif. Le modèle contient déjà `SubcultureEvent` et `BoxLineage`. Il reste à définir précisément le formulaire, l’API, les permissions et l’affichage de l’historique. La stabilisation des codes permettra ensuite de générer les QR codes et de faire ouvrir directement la fiche correspondante depuis la tablette. Le parcours devra garder la recherche manuelle comme solution de secours. Les changements d’armoires pourront alors être journalisés afin de relier les comptages biologiques aux conditions de culture. Les fonctions de consultation viendront compléter ce socle avec des graphiques, des filtres cumulables et des exports CSV ou XLSX. Les règles de partage entre structures devront être définies avant d’activer les accords et les transferts présents dans le modèle. Enfin, le déploiement demandera un choix d’hébergement, PostgreSQL, HTTPS, une gestion rigoureuse des variables d’environnement, des sauvegardes régulières et un test de restauration. Ces étapes garderont la même méthode. Chaque fonction devra partir d’une règle métier confirmée, être protégée selon le rôle et l’organisation, être testée sur le backend et l’interface, puis être documentée pour la reprise du projet.

9. Conclusion

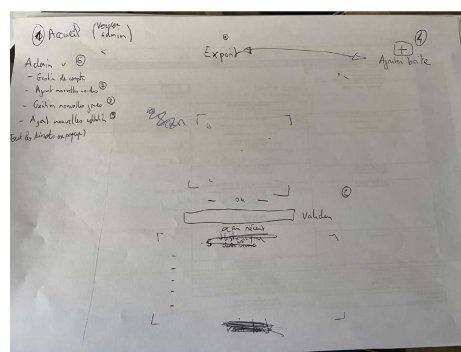
Polyibase est passé d'un besoin exprimé autour de plusieurs fichiers Excel à un premier parcours web relié à une base structurée. Le projet permet déjà de tester la recherche d'une boîte, la consultation de son historique, la saisie d'un relevé sur une interface adaptée à la tablette, l'affichage des zones thermiques et le choix de la langue. Le backend contrôle les organisations et les rôles, puis conserve une trace lors de l'ajout ou de la modification d'un relevé. Le travail sur les données historiques a montré pourquoi cette application demande plus qu'une simple reprise des tableaux existants. Les cellules vides, les changements de périmètre, les noms d'espèces, les températures et les événements de culture doivent être interprétés avant leur import. Le CSV au format long et les notebooks fournissent une base pour ce travail. L'exploration prédictive ouvre aussi une piste, mais elle reste volontairement séparée de l'application tant que sa validation n'est pas suffisante. Les choix techniques préparent la maintenance après le stage. Les domaines sont répartis dans plusieurs applications Django, l'API utilise Django REST Framework, React porte l'interface principale et uv permet de recréer l'environnement Python. Le dépôt, le README, les tests, les données de démonstration et la documentation donnent des points de repère aux futurs développeurs. La suite devra transformer les structures déjà préparées en parcours complets. Le repiquage, les QR codes, les mouvements de boîtes, les exports, les sondes, les accords de partage et le déploiement restent à consolider. Leur réalisation devra conserver la simplicité recherchée au laboratoire et la séparation des données entre aquariums. C'est cette continuité entre biologie, traçabilité et usage quotidien qui donnera à Polyibase sa valeur réelle.

A Maquette papier

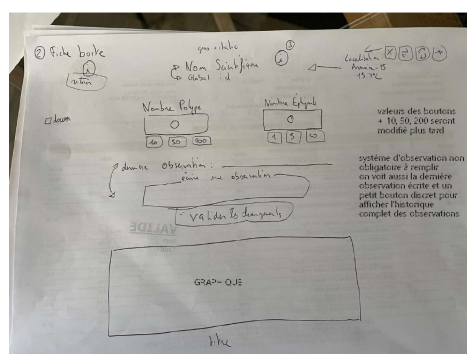
La maquette papier a servi à clarifier les parcours avant de stabiliser l'interface React. Elle présente les premières idées d'écrans autour du pilotage, de la fiche boîte, de la saisie d'un relevé, de l'historique et des zones thermiques. Elle n'est pas utilisée comme rendu final. Elle reste utile pour comprendre le cheminement entre le besoin exprimé au laboratoire et les premiers choix d'interface.



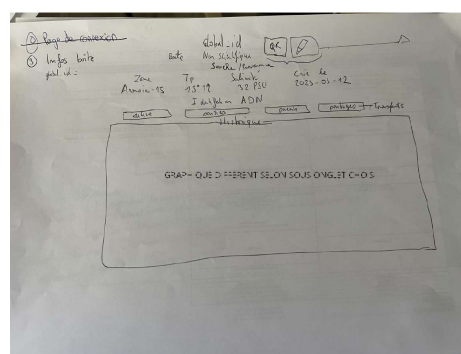
(a) Page 1



(b) Page 2



(c) Page 3



(d) Page 4

Figure 13 – Premières pages de la maquette papier.
Source : docs/maquette_papier/maquette.pdf.

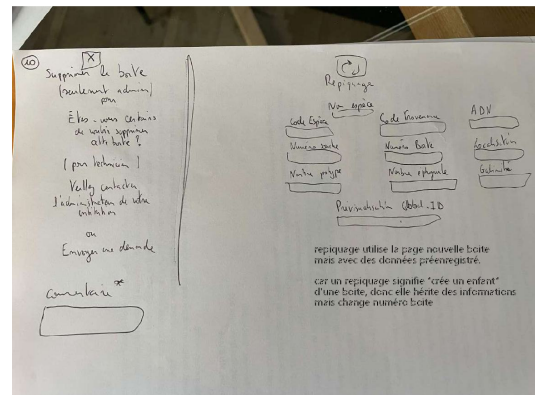
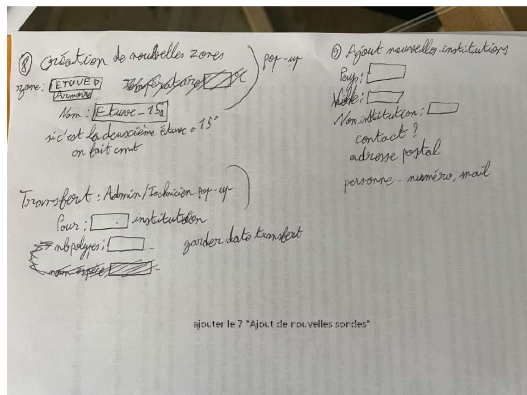
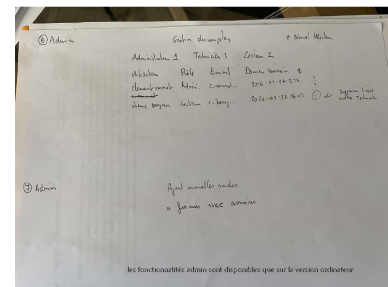
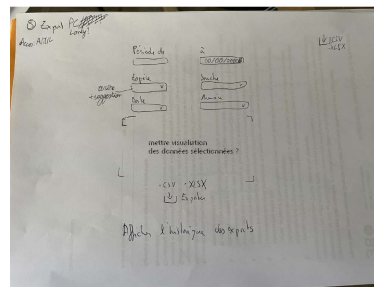
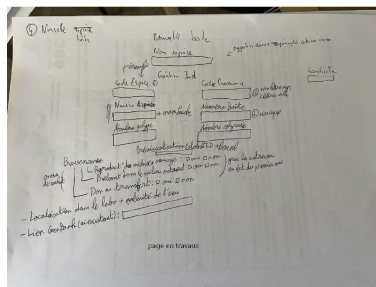


Figure 14 – Pages complémentaires de la maquette papier.
Source : docs/maquette_papier/maquette.pdf.

Source : docs/maquette_papier/maquette.pdf.

Références

- [1] Species360. *ZIMS for Aquariums*. <https://species360.org/products-services/zims-for-aquariums/>.
- [2] Aquarium de Paris. *Médusarium*. <https://aquariumdeparis.com/tours/medusarium/>.
- [3] Jellyfish and Polyps Academy. *Laboratoire*. <https://jellyfishandpolypsacademy.com/laboratoire/>.
- [4] Luxaqua. *LXA Fauniabase*. <https://www.luxaqua.tech/fr/fauniabase/>.
- [5] World Register of Marine Species. *About WoRMS*. <https://www.marinespecies.org/about.php>.
- [6] Helm, R. R. *Evolution and development of scyphozoan jellyfish*. Biological Reviews, 2018. <https://pubmed.ncbi.nlm.nih.gov/29446223/>.
- [7] Fuchs, B. et al. *Regulation of polyp-to-jellyfish transition in Aurelia aurita*. Current Biology, 2014. <https://doi.org/10.1016/j.cub.2013.12.003>.
- [8] Treible, L. M. et Condon, R. H. *Temperature-driven asexual reproduction and strobilation in three scyphozoan jellyfish polyps*. Journal of Experimental Marine Biology and Ecology, 2019. <https://doi.org/10.1016/j.jembe.2019.151204>.
- [9] CNIL. *Les six grands principes du RGPD*. <https://www.cnil.fr/fr/comprendre-le-rgpd/les-six-grands-principes-du-rgpd>.
- [10] CNIL. *Les durées de conservation des données*. <https://www.cnil.fr/fr/passer-laction/les-durees-de-conservation-des-donnees>.